Access-Induced Semantic Divergence in Generated Program Transformations

Anonymous Author(s)
Affiliation
Address
email

Abstract

Access-induced semantic divergence occurs when an operation that appears observational updates latent access state and changes a later externally visible result. We model this pattern as an observation-sensitive deterministic system (OSDS), prove structural properties for a restricted straight-line core, and use the model to build real-package witnesses and OSDS-aware replay checks. The real-code study confirms 20 divergences across 12 unmodified Python packages, with 9 caller-level branch effects and 19/19 applicable mechanism controls eliminating divergence. In a frozen generated-transformation benchmark, five model-task outputs over two underlying package witnesses produced verified OSDS divergence. Ordinary benchmark checks missed all five; OSDS-aware verification caught all five; exact replay reproduced all five; and mechanism-neutralizing controls removed all five while the generated candidate stayed fixed. A prospectively frozen expansion from 7 additional confirmed witnesses produced 42 outputs: 23/42 were task-compliant transformations, 19/42 returned unchanged code, 23/23 compliant transformations preserved OSDS behavior, and 0/42 produced verified OSDS divergence. The results support a bounded conclusion: generated transformations can silently violate access-sensitive semantics on validated witnesses, and a mechanism-specific OSDS model with replay and causal controls provides a verifiable method for detecting and attributing this failure mode.

1 Introduction

Program transformations often treat logging, representation, inspection, caching, and repeated access as behavior-preserving. That treatment is valid when the accessed operation is observational in the semantic sense. It can fail when the operation updates latent state that a later read exposes. We call this pattern access-induced semantic divergence. OSDS adds structure beyond detecting unequal executions: it models the latent state changed by observation-shaped accesses, defines the access-order witness to compare, and specifies neutralizing interventions for attribution.

The phenomenon is related to observer effects and probe effects, but the paper studies a specific sequential mechanism: an observation-shaped access mutates latent access state, and a later program result changes. This matters for generated code because requested edits are often local and plausible. A model may add instrumentation, memoize a representation, or simplify repeated access while preserving ordinary tests that do not vary observation order.

We study this mechanism through three evidence levels. First, phenomenon breadth: the real-code oracle confirms 20 divergences across 12 unmodified Python packages, with 9 caller-level effects and 19/19 applicable real-code mechanism controls eliminating divergence. Second, generated-code failures: the frozen primary benchmark contains five verified model-task OSDS failures over two underlying package witnesses, pytest and PyYAML, and all five were silent under the benchmark or-

Submitted to NeurIPS 2026 Workshop on AI for Verifiable Coding. Do not distribute.

```
Original shape:
  with catching_logs(handler, level):
      result = subject(logger, handler)

Generated instrumentation shape:
  logger.debug("diagnostic")
  with catching_logs(handler, level):
      result = subject(logger, handler)

State flow:
  inserted diagnostic access -> logging/handler state changes
  -> later catching_logs observes changed state
  -> caller-visible result differs

Verification:
  ordinary check PASS | OSDS preservation FAIL | neutralized replay PASS
```

Figure 1: Compact pytest failure schema. The inserted log call looks like observation. In the verified Terra and Luna rows, it mutates logging/handler state that is read later. The same generated candidates pass the ordinary benchmark check, fail OSDS preservation, and pass after the diagnostic logger is isolated from the captured handler hierarchy.

dinary checks. Third, prospective limitation: a prospectively frozen expansion built from previously confirmed unused witnesses produced 42 model outputs, of which 23 were task-compliant transformations, 19 returned unchanged code, 23/23 compliant transformations were OSDS-preserving, and 0/42 produced verified OSDS divergence.

The empirical claim is bounded. We do not estimate ecosystem prevalence, and the three tested Codex task-model configurations are not independent model providers. The strongest generated-code evidence is the causal chain in the primary benchmark: five generated transformations passed ordinary checks, failed OSDS-aware verification, reproduced under exact replay, and disappeared under mechanism-neutralizing controls while the generated candidate stayed fixed. These are five model-task outcomes over two underlying package witnesses, not a frequency estimate.

Contributions. We make four contributions. First, we give an OSDS mechanism model and structural properties for access-state-carrying sequential systems. Second, we give a real-code witness methodology and report 20 confirmed divergences across 12 packages, including 9 caller-level branch effects. Third, we evaluate frozen generated transformations and identify five silent model-task OSDS failures over two package witnesses. Fourth, we combine mechanism-neutralizing causal attribution with a prospectively frozen null expansion that separates task compliance from semantic preservation.

2 Motivating Real-Package Example

Figure 1 shows the shortest verified failure family from the primary study. The task asked for behavior-preserving instrumentation around a pytest logging witness. The generated edit inserted diagnostic logging that looked observational. In the witness, that access interacted with the captured handler hierarchy used by `catching_logs`; the later handler-level behavior changed.

The example gives the intuition before the formal notation. The generated transformation did not crash, did not fail the ordinary smoke check, and did not look like a destructive update at the edit site. The divergence appeared only when the evaluation compared the later visible result across the access order that the witness exposes. In the causal-control replay, the exact generated files were reused. The only change was the witness environment: diagnostic logging was isolated from the handler hierarchy responsible for the latent mutation. Under that neutralized mechanism, the OSDS divergence disappeared.

This is a generated transformation failure, not a defect claim about pytest. It shows that a preservation checker for generated edits needs to model the latent state that observation-shaped accesses can affect.

Stage	Baseline path	Transformed path	OSDS question
Initialize	matched object	matched object	same visible setup
Observe	no extra access	observation-shaped access	can latent state drift?
Read	later visible read	later visible read	does exposed value change?
Classify	ordinary check	OSDS comparison	is divergence access-induced?

Figure 2: OSDS evaluation compares access-order variants that ordinary single-order checks may not exercise.

3 Observation-Sensitive Deterministic Systems

An OSDS configuration contains a semantic value $x = (b, a, d)$ and an accumulator y. The component b is stable, a is an access count, and d is latent drift. A read exposes a value and updates access state:

$$\mathsf{read}(b, a, d, y) = ((b, a + 1, r(a, d)), y + f(b, a, d)).$$

An observation exposes no additive value but can update latent drift:

$$\mathsf{obs}(b, a, d, y) = ((b, a, g(d)), y).$$

A program body is a deterministic fold over a finite operation sequence $\pi \in \{\mathsf{read}, \mathsf{obs}\}^*$. A cap C maps the final body configuration to the externally visible result.

For a fixed initial configuration cfg_0, body sequence π, and cap C, let $[\![\pi]\!]_C(\mathsf{cfg}_0)$ denote the final visible output. Given a multiset of operations M and an admissible ordering set $\Pi(M)$, semantic divergence is

$$\Delta_C(M, \mathsf{cfg}_0) = \max_{\pi \in \Pi(M)} [\![\pi]\!]_C(\mathsf{cfg}_0) - \min_{\pi \in \Pi(M)} [\![\pi]\!]_C(\mathsf{cfg}_0).$$

A transformation is OSDS-preserving for a witness when the baseline and candidate agree under the witness's OSDS relation. In the benchmark, this relation is implemented as a package-specific metamorphic oracle and then manually reviewed for access-induced attribution.

4 Structural Results

The structural core is a straight-line deterministic template that supports the witness construction and causal-control interpretation. A read and observation commute locally only when both the latent-state update and exposed contribution agree across the two orders; this gives the OSDS oracle a direct semantic criterion for when an access reorder is locally inert (Appendix A.2). Within that scope, fixed-order execution is deterministic: induction over the operation sequence gives a unique next configuration at each step. Two zero-divergence conditions follow directly. If $g(d) = d$, observations are inert and can be moved without changing the reduced read sequence. If $f(b, a, d) = h(b)$, reads ignore access count and drift, so every ordering with the same number of reads produces the same body accumulator.

A linear cap preserves body-level divergence. For $C(y) = \alpha y + \beta$ with $\alpha \neq 0$, $y_1 \neq y_2$ implies $C(y_1) - C(y_2) = \alpha(y_1 - y_2) \neq 0$. This proposition is the only cap result used as an unbounded structural claim. Nonlinear amplification and broad degree laws are treated as bounded computational findings.

The local commutation proposition characterizes exact equality of one adjacent RO/OR swap. The appendix then states a restricted adjacent-swap result for the checked positive affine fragment, where read and observation drift updates are additive and commute: under those stronger assumptions, repeated nondecreasing swaps push extrema toward boundary orderings. The appendix also states the restricted range-degree result supported by exact rational checks: for checked compositional OSDS cases $m = 1..4$, cap degree $d = 1..5$, and $L = 2..15$, extremal-order interpolation detects degree $2d$. This is not a general theorem for all OSDS programs.

5 Real-Code Study

The real-code study uses static screening only as a review queue. The screen covered 73 PyPI packages and 278 reviewed MEDIUM/HIGH findings. Manual review labeled 203 likely true positives

Real-code evidence item	Count
Reviewed static findings	278
Executable harnesses	39
Confirmed divergences	20
Confirmed packages	12
Caller-level wrappers changing downstream execution	9
Mechanism controls eliminating divergence	19/19

Table 1: Real-code OSDS evidence. Static screening is a review queue; executable harnesses and controls provide the semantic evidence.

Model	Exec.	Pres.	Ord. bug	Invalid	OSDS fail	Silent OSDS
gpt-5.6-sol	26	23	3	0	0	0
gpt-5.6-terra	24	18	4	2	2	2
gpt-5.6-luna	24	17	4	2	3	3

Table 2: Primary frozen coding-agent results. Exec. is executable outputs; Pres. means behavior preserved under the benchmark evaluation, not merely executable; Ord. bug means ordinary programming bug. Silent OSDS means a manually verified OSDS divergence for which the benchmark ordinary check passed.

and 75 likely false positives, for reviewed precision 0.7302. A rebuilt exact-version source snapshot contained 4,383 analyzable classes, and a deterministic sample of 200 unflagged classes found 0 likely missed matches. These numbers are audit evidence rather than prevalence evidence.

Executable confirmation is the decisive filter. The oracle selected 60 candidates, constructed 39 executable package-shaped harnesses, and confirmed 20 divergences across 12 unmodified packages: httpcore, PyYAML, pytest, markdown, more-itertools, docutils, beautifulsoup4, boltons, cerberus, dnspython, h11, and anyio. The divergences include stream materialization, identity-cache reuse, handler-level mutation, reference-registry mutation, cursor advance, destructive tree extraction, cache recency/statistics, validation-error population, and buffer consumption.

Nine confirmed divergences were lifted into caller-level wrappers that changed branch labels and downstream consequences. Nineteen controls eliminated divergence under fresh-object, reset-between, or pure-observation interventions. These controls show that the detected differences depend on the access-induced mechanism named by the witness.

6 Generated Transformation Study

The primary benchmark was frozen before model execution. It contains 13 base tasks and 26 normal/warned variants from 9 packages and 9 unique witnesses. The six transformation families are instrumentation, caching/materialization, refactoring, repeated-access cleanup, access reordering, and debugging/inspection. Each generation task was fresh and projectless. The model saw only the frozen prompt, with no OSDS labels, oracle output, witness labels, benchmark explanation, or previous responses.

Replay treats generated text as data. Raw responses are saved exactly, code is extracted, ordinary smoke checks are run, and then OSDS-aware checks compare baseline and candidate behavior under the witness relation. Manual review classifies non-preserved rows as verified semantic divergence, ordinary programming bug, invalid patch, environment failure, oracle issue, or unclear.

The benchmark includes hidden-observation rows, where an access looks observational in the requested edit and reveals its effect only through later state, and expected-access-sensitive rows, where consumption or mutation should be visible from the code shape. The five verified OSDS failures are five model-task outcomes over two underlying package witnesses. Terra failed on two pytest instrumentation rows. Luna failed on those two pytest rows and one PyYAML caching/materialization row. Sol preserved the verified-OSDS rows on which Terra or Luna diverged. Expected-access-sensitive rows produced ordinary bugs or invalid patches, with no verified OSDS divergence.

Family	Intervention	Rows	Result
pytest instrumentation	isolate diagnostic logger from captured handler hierarchy	4	4/4 removed
PyYAML caching	clear representer identity cache after observation	1	1/1 removed

Table 3: Causal controls. Under the original witness all five failures reproduced; under mechanism-neutralizing controls all five OSDS divergences disappeared.

Model	Outputs	Task-compliant	Unchanged	Ordinary pass	Ver. OSDS
gpt-5.6-sol	14	8	6	14	0
gpt-5.6-terra	14	8	6	14	0
gpt-5.6-luna	14	7	7	14	0
Total	42	23	19	42	0

Table 4: Prospective expansion with task compliance separated from semantic preservation. Unchanged outputs are OSDS-preserving but are not counted as task-compliant transformations.

Self-assessment was secondary. Across Sol, Terra, and Luna, there were zero false YES preservation claims on verified OSDS divergences. Conservative NO responses were common, so self-assessment should not replace execution replay.

7 Causal Attribution and Prospective Expansion

The causal-control experiment replays the exact generated files for the five verified failures. The candidate source remains unchanged. The intervention changes only the witness mechanism that caused access-induced divergence.

This supports the access-induced attribution. The failures are mechanism-dependent generated-code defects: ordinary checks pass, OSDS checks fail, and targeted neutralization removes divergence while preserving the generated candidate.

After the causal phase, 11 previously confirmed unused real-code witnesses were audited prospectively under stated eligibility criteria. Seven met those criteria and were frozen before model execution, producing 14 normal/warned variants from boltons, dnspython, and h11. Sol, Terra, and Luna were run on these exact prompts as fresh projectless tasks. Self-assessment was skipped for the expansion.

The prospective benchmark produced no new verified OSDS divergences. The corrected interpretation is narrower than a 42/42 transformation success claim: 23/42 outputs performed a nontrivial task-compliant transformation, 19/42 returned unchanged code, and 23/23 task-compliant outputs were OSDS-preserving. The result constrains failure incidence among these frozen responses, supports reporting the primary failures as reproducible examples rather than as a prevalence estimate, and provides evidence against treating the selected witness construction as mechanically failure-producing.

8 Related Work

Observer effects and Heisenbugs. Heisenbugs and probe effects describe programs whose behavior changes under observation or debugging. Musuvathi et al. discuss debugging statements affecting concurrent interleavings in systematic testing [Musuvathi et al., 2008]. Sallinger, Weissenbacher, and Zuleger formalize Heisenbugs as hyperproperties and analyze causes including probe effects and nondeterminism [Sallinger et al., 2026]. OSDS is narrower: it models sequential access-state mechanisms where observation-shaped operations mutate latent state and later reads expose the difference.

Semantic preservation and equivalence. Compiler verification and translation validation study semantic preservation between source and transformed programs [Pnueli et al., 1998, Leroy, 2009]. OSDS uses the same preservation motivation but focuses on generated source-level transformations

over library objects whose read and observation operations carry latent state. The oracle compares access-order behavior rather than proving full program equivalence.

Metamorphic testing. Metamorphic testing addresses oracle problems by checking relations across related executions [Chen et al., 1998, 2018]. The OSDS-aware oracle is a specialized metamorphic relation: the follow-up execution changes the placement of an observation-shaped access while keeping witness setup matched. Generic metamorphic testing supplies the testing pattern; OSDS supplies the mechanism and classification boundary.

Current generated-transformation benchmarks. Differential fuzzing has recently been applied to LLM-generated code refactorings, showing that generated refactors can be functionally nonequivalent and that existing tests can miss a substantial fraction of those cases [Dristi and Dwyer, 2026]. SWE-Refactor moves refactoring evaluation to repository-level Java changes mined from real projects, emphasizing realistic context and compound refactorings [Xu et al., 2026]. VERINA benchmarks verifiable code generation in Lean across code, specification, and proof generation tasks [Ye et al., 2025]. Viverra studies text-to-code with generated assertions and bounded verification, surfacing verified properties alongside generated C programs [Wu et al., 2026]. Metamorphic transformations have also been used to diagnose memorization in LLM-based program repair; we use that line only as related evidence that semantics-preserving transformations can support stronger code-evaluation protocols [Koning et al., 2026].

OSDS mechanism attribution. Generic differential and metamorphic testing can establish that two executions differ under a chosen relation. OSDS contributes a sequential semantic model for objects whose reads and observations interact through latent access state. That structure determines which access reorderings form meaningful witnesses and supports mechanism-specific controls. The causal interventions test whether a detected divergence disappears when the access-induced mechanism is neutralized while the generated candidate remains unchanged, giving mechanism attribution alongside behavioral detection.

Generated code and repair verification. HumanEval and Codex made execution-based evaluation central for code generation [Chen et al., 2021]. SWE-bench extends evaluation to issue-resolution patches in real repositories [Jimenez et al., 2024]. Automated program repair has long shown that patches can overfit tests [Le Goues et al., 2012, Smith et al., 2015]. This paper studies a different preservation failure: a generated transformation can pass the ordinary benchmark tests while violating an access-state relation that those tests do not exercise.

9 Limitations

The coding-agent benchmark is small and correlated by witness and package. Counts are benchmark outcomes, not prevalence estimates. The five verified primary failures are five model-task failures over two underlying package witnesses. The prospective expansion added seven witnesses and found zero new verified OSDS divergences.

The three model labels are Codex task-model configurations reported by the Codex runtime. They were collected through fresh projectless Codex tasks rather than the OpenAI API, and temperature/seed were not exposed. The study is Python-focused. Manual review remains part of verified OSDS classification. The formal results apply to the stated straight-line deterministic core; broader analyzer and range-degree claims remain empirical or bounded computational evidence.

The intended positive impact is better preservation testing for generated code in libraries where observation-shaped accesses can mutate hidden state. The main risk is that the same examples could encourage superficial transformations that pass ordinary tests while changing access-sensitive behavior. We mitigate this risk by releasing bounded witnesses, replay scripts, and mechanism-neutralizing controls rather than a general attack system, and by framing the results as verification evidence rather than deployment guidance.

10 Conclusion

Access-induced semantic divergence is a concrete preservation risk for generated transformations on access-state-carrying software objects. The OSDS model identifies the mechanism, the real-code study confirms package witnesses, and the coding-agent benchmark shows five silent generated

failures under ordinary checks. Exact replay plus mechanism-neutralizing controls reproduced and removed all five primary failures. The prospective expansion found no new OSDS divergences and exposed substantial unchanged-output behavior. The resulting assessment is bounded: OSDS-aware verification provides a reproducible method for detecting and attributing a semantic failure mode that ordinary checks missed in this benchmark, without supporting broad claims of model unsafety.

References

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.

T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, 1998.

T. Y. Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018. doi: 10.1145/3143561.

Simantika Bhattacharjee Dristi and Matthew B. Dwyer. A differential fuzzing-based evaluation of functional equivalence in LLM-generated code refactorings. *CoRR*, abs/2602.15761, 2026. doi: 10.48550/arXiv.2602.15761.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=VTF8yNQM66`.

Milan De Koning, Ali Asgari, Pouria Derakhshanfar, and Annibale Panichella. A metamorphic testing approach to diagnosing memorization in LLM-based program repair. *CoRR*, abs/2604.21579, 2026. doi: 10.48550/arXiv.2604.21579.

Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi: 10.1145/1538788.1538814.

Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, and Iulian Neamtiu. Finding and reproducing heisenbugs in concurrent programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation*, pages 267–280, 2008.

Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems*, volume 1384 of *Lecture Notes in Computer Science*, pages 151–166. Springer, 1998. doi: 10.1007/BFb0054170.

Sarah Sallinger, Georg Weissenbacher, and Florian Zuleger. A formalization of heisenbugs and their causes in terms of hyperproperties. *Software and Systems Modeling*, 2026. doi: 10.1007/s10270-025-01345-7.

Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. Is the cure worse than the disease? overfitting in automated program repair. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 532–543, 2015. doi: 10.1145/2786805.2786825.

Haoze Wu, Rocky Klopfenstein, Keith Farkas, and Nina Narodytska. Viverra: Text-to-code with guarantees. *CoRR*, abs/2605.14972, 2026. doi: 10.48550/arXiv.2605.14972.

Yisen Xu, Jinqiu Yang, and Tse-Hsun Chen. SWE-refactor: A repository-level benchmark for real-world LLM-based code refactoring. *CoRR*, abs/2602.03712, 2026. doi: 10.48550/arXiv.2602.03712.

Zhe Ye, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song. VERINA: Benchmarking verifiable code generation. *CoRR*, abs/2505.23135, 2025. doi: 10.48550/arXiv.2505.23135.

A Formal Details

A.1 State and transitions

An OSDS state is $s = (b, a, d, y)$ with base b, access count a, latent drift d, and accumulator y. A read transition is deterministic:

$$R(s) = (b, a + 1, r(a, d), y + f(b, a, d)).$$

An observation transition is deterministic:

$$O(s) = (b, a, g(d), y).$$

For a finite sequence $\pi = o_1 \dots o_n$, composition is $[\![\epsilon]\!](s) = s$ and $[\![\pi o]\!](s) = o([\![\pi]\!](s))$. Branch evaluation is a deterministic predicate $B(C([\![\pi]\!](s)))$ over the capped visible result or final configuration. Branch divergence occurs when two admissible orderings produce different branch labels or downstream consequences.

A.2 Local read-observation commutation

Proposition. Let $s = (b, a, d, y)$ and let R and O be the OSDS read and observation transitions defined above. Then

$$O(R(s)) = (b, a + 1, g(r(a, d)), y + f(b, a, d))$$

and

$$R(O(s)) = (b, a + 1, r(a, g(d)), y + f(b, a, g(d))).$$

Thus R and O commute at state s, in the sense of equality of the full resulting OSDS state, if and only if

$$g(r(a, d)) = r(a, g(d)) \quad \text{and} \quad f(b, a, d) = f(b, a, g(d)).$$

Proof. Expanding the read first gives $R(s) = (b, a + 1, r(a, d), y + f(b, a, d))$. Applying O to that state leaves b, $a+1$, and the accumulator unchanged and updates latent drift by g, giving $O(R(s)) = (b, a + 1, g(r(a, d)), y + f(b, a, d))$. Expanding the observation first gives $O(s) = (b, a, g(d), y)$. Applying R to that state increments the same access count, updates drift by $r(a, g(d))$, and adds $f(b, a, g(d))$, giving $R(O(s)) = (b, a+1, r(a, g(d)), y+f(b, a, g(d)))$. The b and $a+1$ components are equal in both results, so equality of the full states is exactly equality of the latent drift components and accumulator increments shown above.

An adjacent read-observation reorder can affect OSDS state only at states where this local condition fails in latent state, exposed contribution, or both. Local noncommutation is a necessary source of order sensitivity at the swapped pair; downstream execution and the cap determine whether that local difference becomes externally visible. This is why the OSDS-aware metamorphic relation is not an arbitrary input perturbation: it compares executions that differ in the placement of observation-shaped accesses, and the commutation condition characterizes when such a local reorder is inert in the OSDS core. When the condition fails, the witness has a mechanism-grounded reason to test the reordered execution, while the oracle remains a witness-specific check rather than a completeness theorem for all semantic divergence.

A.3 Semantic divergence

Given operation multiset M, admissible orderings $\Pi(M)$, initial state s_0, and deterministic cap C, define $V_C(\pi, s_0) = C([\![\pi]\!](s_0))$. The access-order range is $\Delta_C(M, s_0) = \max_{\pi \in \Pi(M)} V_C(\pi, s_0) - \min_{\pi \in \Pi(M)} V_C(\pi, s_0)$ for finite $\Pi(M)$. A transformation preserves the witness when baseline and candidate have equal OSDS-observable results under the witness relation.

A.4 Fixed-order determinism

Proposition. For fixed s_0, fixed π, deterministic R, deterministic O, and deterministic C, $V_C(\pi, s_0)$ is unique. **Proof.** Induct on $|\pi|$. The empty sequence returns s_0. For πo, the induction hypothesis gives a unique intermediate state and $o \in \{R, O\}$ maps it to a unique next state. The deterministic cap gives a unique output.

A.5 Zero divergence under no observation-induced mutation

Proposition. If $g(d) = d$ and observations expose no additive value, moving observations within a fixed multiset does not change the body accumulator or final latent state. **Proof.** Each observation is the identity on (b, a, d, y) and contributes no value to the accumulator. Delete observations from any admissible ordering; the remaining read sequence has the same length and order because all reads are identical. The same deterministic reads are therefore executed from the same state and contribute the same accumulator terms. Re-inserting identity observations at arbitrary positions leaves the accumulator and final latent state unchanged.

A.6 Zero divergence under access-insensitive reads

Proposition. If $f(b, a, d) = h(b)$, every ordering with L reads has the same body accumulator $Lh(b)$. **Proof.** Every read contributes the same exposed value for fixed b regardless of access count or drift. Observations may change d, but d is ignored by f. A deterministic cap depending only on the accumulator and final counts preserves equality.

A.7 Linear-cap preservation

Proposition. Let $C(y) = \alpha y + \beta$ with $\alpha \neq 0$. If body accumulators $y_1 \neq y_2$, then $C(y_1) \neq C(y_2)$. **Proof.** $C(y_1) - C(y_2) = \alpha(y_1 - y_2)$, which is nonzero under exact arithmetic.

A.8 Adjacent-swap extrema under monotone access drift

Restricted lemma. Consider the checked positive affine fragment with identical reads, identical observations, deterministic updates

$$r(a, d) = d + \delta, \qquad g(d) = d + \eta,$$

where $\delta \geq 0$, $\eta \geq 0$, and read contribution $f(b, a, d) = b + ad$ with nonnegative access count a. Observations do not change a or y, reads increase a by one, and the final cap is monotone in the accumulated read contribution. For any adjacent pair RO in an otherwise fixed ordering, replacing it by OR cannot decrease the contribution of the swapped read or any later read. **Proof.** Let the common prefix before the pair produce state (b, a, d, y). In the local order RO, the read contributes $b + ad$ and updates drift to $d + \delta$; the following observation updates drift to $d + \delta + \eta$. Thus the post-pair state is $(b, a + 1, d + \delta + \eta, y + b + ad)$. In the local order OR, the observation first updates drift to $d + \eta$; the read then contributes $b + a(d + \eta)$ and updates drift to $d + \eta + \delta$. Thus the post-pair state is $(b, a + 1, d + \eta + \delta, y + b + a(d + \eta))$. The two orders therefore enter the suffix with the same b, the same access count $a + 1$, and the same drift because $d + \delta + \eta = d + \eta + \delta$. The OR accumulator is no smaller since $a\eta \geq 0$. The suffix sees the same latent state in both executions and starts from an accumulator no smaller in the OR run, so later read contributions and the final monotone capped value cannot be smaller. Repeated adjacent swaps can move each observation left across neighboring reads without decreasing value, so an ordering with all observations before all reads is a maximum among orderings reachable by these swaps. The reverse argument moves observations right across reads without increasing value, giving the all-reads-before-observations boundary as a minimum. **Scope.** The lemma is limited to this checked positive affine straight-line fragment with additive commuting read and observation drift updates and monotone caps. It is not claimed for arbitrary Python programs, noncommuting read/observation drift updates, or nonmonotone observation effects.

A.9 Restricted 2d range-degree statement

Bounded exact computational result. For the compositional OSDS cap family implemented in the exact-rational validation scripts, with $m = 1..4$, cap degree $d = 1..5$, and $L = 2..15$, exhaustive checks on small feasible grids plus extremal-order interpolation detect access-order range degree $2d$. **Derivation.** In the checked family, the body-range term is $\eta m L(L-1)/2$. The compositional cap contributes $d-1$ final read factors, each quadratic in L. The leading degree is therefore $2+2(d-1) = 2d$. Exact rational scripts verify the extremal formula against enumerated feasible orderings and holdout values on the stated bounded domain. **Scope.** This is a restricted exact computational result for the named family and grid. It is not a general theorem for all OSDS programs.

B Benchmark Details

Primary tasks are in `benchmark/tasks.jsonl` under the agent-behavior study tree. Primary prompt files are in that tree's `prompts/` directory. The pre-model manifest and raw Codex task-model responses are in `external_collection/` and `external_collection/responses/`.

Manual review rows are in the primary study analysis directory as the 20260813 manual-review JSONL. The matching cross-model summary is stored beside it as JSON and Markdown. Prospective tasks, prompts, and raw responses are under `benchmark_expansion/`. Exact prospective replay outputs are in the three `*expansion-exact-20260813Tcutscope` result directories.

The prospective task-compliance audit is `analysis/prospective_task_compliance.csv`. It separates `task_compliance`, `ordinary_correctness`, and `osds_preservation` for all 42 prospective outputs. The supplement artifact index maps these shortened descriptions to exact relative paths.

C Reproducibility

The reviewer supplement contains a relative-path artifact index, frozen prompts, raw response JSONLs, replay summaries, causal-control scripts, real-code oracle metadata, and expected outputs. It excludes virtual environments, caches, authentication material, and personal absolute paths. Replays require Python 3.11 or newer with pytest and the package versions named by the witness manifests. The supplement `REPRODUCE.md` gives exact commands for the primary replay, prospective replay, causal-control replay, and table reproduction.

All artifact replays are CPU-only Python executions and do not require a GPU, accelerator runtime, cloud worker, model credential, or model-provider API call. A pre-submission reproduction check on Windows 11 with Python 3.14.4, a 6-core/12-thread AMD Ryzen 5 4600H CPU, and 7.4 GiB RAM measured 1.12 s for a baseline single-task replay, 1.22 s for a frozen primary single-task replay, 1.13 s for a frozen prospective single-task replay, 3.49 s for the five-case causal-control replay, 0.57 s for the real-code witness validation command, and 0.13 s for the summary-table command. Exact peak memory was not logged; the commands operate on small JSONL, CSV, and Python files, and the packaged supplement is below 1 MB. The earlier Codex task-model generation phase is not reproduced by the supplement. Its inference hardware, accelerator hours, and hidden service-side memory were not exposed by the Codex task interface, and preliminary or failed collection attempts used additional model calls outside the replay package.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the OSDS mechanism, bounded scope, primary causal result, and prospective accounting separately.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper has a dedicated Limitations section covering benchmark size, model-label correlation, Python scope, manual review, and formal scope.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: The formal claims are stated with assumptions; complete proofs for the local read-observation commutation proposition, other propositions, and restricted adjacent-swap lemma are in the appendix, and the 2d claim is labeled as a bounded exact computational result.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The appendix and supplement disclose frozen prompts, raw responses, replay commands, causal controls, artifact indices, and expected outputs needed to reproduce the reported results from frozen responses.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The anonymized supplement provides code, data, frozen raw responses, and a REPRODUCE.md with exact relative-path commands for the supplied artifacts.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The study reports task construction, frozen prompt conditions, replay model labels, ordinary and OSDS checks, causal controls, and prospective task-compliance accounting.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [N/A]

   Justification: The reported results are deterministic replay counts over frozen artifacts and curated witnesses; the paper does not make sampling or statistical-significance claims.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The reproducibility appendix and supplement state the CPU-only replay worker, Python and OS version, RAM, measured wall-clock times for the replay commands, supplement size, and the limitation that Codex generation compute was service-side and not exposed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The study uses public packages and frozen generated text artifacts, releases anonymized supplement material, and reports no human-subject or sensitive-data collection.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Limitations section discusses positive verification uses and negative risks from silent preservation failures or superficial transformations, plus mitigations through bounded witnesses, replay scripts, and mechanism-neutralizing controls.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper releases no model, scraped dataset, or dual-use trained artifact requiring special misuse safeguards.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Existing packages and benchmark lineages are cited; supplement LICENSES.md records package versions, license names or classifiers, generated-response status, and official-template provenance without redistributing full third-party package source trees.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper introduces an anonymized benchmark and supplement; the artifact index, manifests, prompts, frozen responses, reproduction notes, and LICENSES.md document the released artifacts and terms.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve human subjects, so IRB or equivalent approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLM use is central to the generated-transformation study and is described through the Codex task-model configurations, frozen prompts, raw responses, and replay protocol.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.